

Návrhový vzor Chain of Responsibility

Cíl úkolu

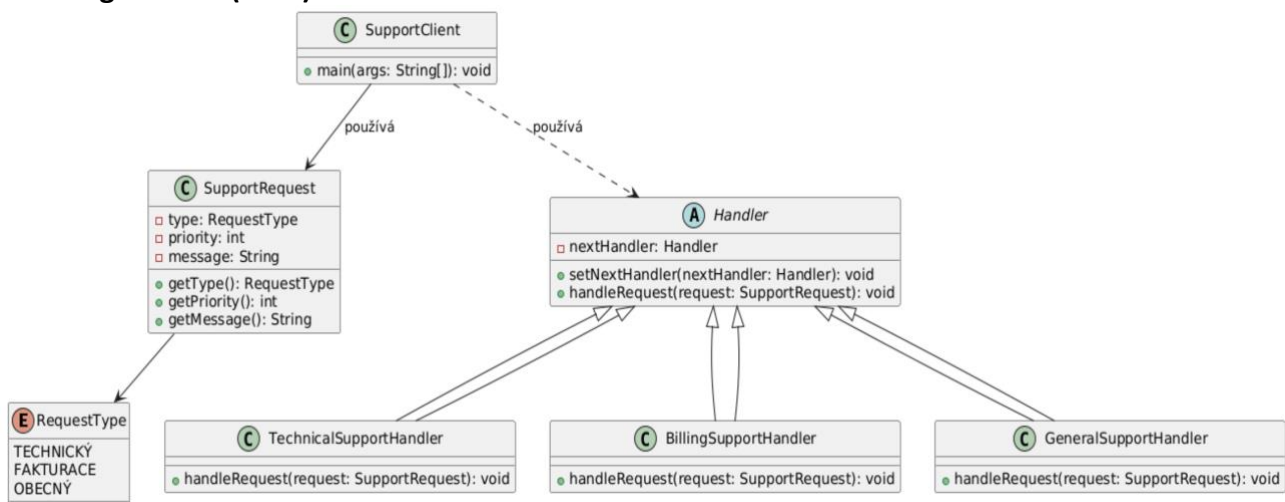
Cílem tohoto cvičení je seznámit se s návrhovým vzorem **Chain of Responsibility** v jazyce Java. Vaším úkolem bude navrhnout a implementovat systém pro zpracování požadavků na podporu. Každý požadavek bude mít určitou prioritu a typ, a systém podpory zajistí, že bude předán správnému oddělení nebo úrovni podpory. Požadavky, které jedno oddělení nemůže zpracovat, budou předány dalšímu oddělení v řetězci.

Popis systému

Vaše implementace bude obsahovat následující třídy:

1. **RequestType** – Výčtový typ definující typy požadavků, které mohou být zpracovány. Typy budou zahrnovat:
 - TECHNICKÝ – požadavky technického charakteru.
 - FAKTURACE – požadavky týkající se fakturace.
 - OBECNÝ – obecné požadavky, které nepatří do konkrétní kategorie.
2. **SupportRequest** – Třída reprezentující požadavek na podporu. Obsahuje typ požadavku (například TECHNICKÝ nebo FAKTURACE), prioritu a zprávu, která popisuje problém. Poskytuje metody pro získání jednotlivých hodnot (getType, getPriority, getMessage).
3. **Handler** – Abstraktní třída, která definuje základní metodu handleRequest pro zpracování požadavků a obsahuje referenci na dalšího handlera v řetězci. Handler obsahuje metodu setNextHandler, která umožňuje vytvořit řetězec zodpovědnosti.
4. **TechnicalSupportHandler** – Konkrétní handler pro technickou podporu. Tento handler zpracovává požadavky typu TECHNICKÝ s prioritou 2 a vyšší.
5. **BillingSupportHandler** – Konkrétní handler pro podporu fakturace. Tento handler zpracovává požadavky typu FAKTURACE s prioritou 2 a vyšší.
6. **GeneralSupportHandler** – Obecný handler, který zpracovává všechny ostatní požadavky nebo požadavky s nízkou prioritou, které nebyly zpracovány žádným předchozím handlerem.
7. **SupportClient** – Třída, která vytváří řetězec handlerů a odesílá požadavky na podporu. Obsahuje metodu main, ve které jsou vytvořeny různé typy požadavků a předány do systému podpory.

Diagram tříd (UML):



Požadavky na implementaci

1. **Implementujte jednotlivé třídy podle UML diagramu:**
 - Každá třída (SupportRequest, TechnicalSupportHandler, BillingSupportHandler, GeneralSupportHandler, SupportClient) by měla implementovat metody uvedené v diagramu a simulovat své funkce.
2. **Implementujte třídu SupportRequest:**
 - Tato třída bude obsahovat proměnné type, priority a message a odpovídající metody pro jejich získání.
3. **Implementujte třídu Handler:**
 - Definujte abstraktní metodu handleRequest, která zpracuje požadavek, a metodu setNextHandler, která nastaví odkaz na dalšího handlera v řetězci.
4. **Implementujte třídy TechnicalSupportHandler, BillingSupportHandler a GeneralSupportHandler:**
 - **TechnicalSupportHandler** – zpracovává požadavky typu TECHNICKÝ s prioritou 2 a vyšší.
 - **BillingSupportHandler** – zpracovává požadavky typu FAKTURACE s prioritou 2 a vyšší.
 - **GeneralSupportHandler** – zpracovává všechny ostatní požadavky, které nebyly zpracovány předchozími handlers.
5. **Vytvořte hlavní třídu SupportClient:**
 - Vytvořte řetězec handlerů: TechnicalSupportHandler - > BillingSupportHandler -> GeneralSupportHandler.
 - Vytvořte různé typy požadavků a předávejte je řetězcem handlerů k vyřešení.

Specifikace metod

- **RequestType**
 - Definujte výčtové hodnoty TECHNICKÝ, FAKTURACE, OBECNÝ.
- **SupportRequest**
 - Konstruktor a metody getType, getPriority, getMessage.
- **Handler**
 - setNextHandler(nextHandler: Handler): void – Nastaví dalšího handlera v řetězci.
 - handleRequest(request: SupportRequest): void – Abstraktní metoda pro zpracování požadavků.
- **TechnicalSupportHandler, BillingSupportHandler, GeneralSupportHandler**
 - Implementujte metodu handleRequest v každé třídě podle pravidel popsaných výše.
- **SupportClient**
 - main(args: String[]): void – Vytvořte handlerové třídy a řetězec zodpovědnosti, otestujte zpracování požadavků.

```
// Klientská třída
public class SupportClient {
    public static void main(String[] args) {
        // Vytvoření handlerů
        Handler technicalSupport = new TechnicalSupportHandler();
        Handler billingSupport = new BillingSupportHandler();
        Handler generalSupport = new GeneralSupportHandler();

        // Vytvoření řetězce zodpovědnosti
        technicalSupport.setNextHandler(billingSupport);
        billingSupport.setNextHandler(generalSupport);

        // Vytvoření požadavků
        SupportRequest techRequest = new SupportRequest(RequestType.TECHNICKÝ, priority: 2, message: "Problém s připojením k serveru");
        SupportRequest billingRequest = new SupportRequest(RequestType.FAKTURACE, priority: 3, message: "Dotaz na fakturu");
        SupportRequest lowPriorityRequest = new SupportRequest(RequestType.OBECNÝ, priority: 1, message: "Obecný dotaz");

        // Zpracování požadavků
        technicalSupport.handleRequest(techRequest);
        technicalSupport.handleRequest(billingRequest);
        technicalSupport.handleRequest(lowPriorityRequest);
    }
}
```